MIMU4844/MIMU48XC

Integration Guide

Revision 1.5

Registered Office:
GT Silicon Pvt Ltd
LIG 1398, Avas Vikas 3,
PO – NSI, Kalyanpur,
Kanpur (UP), India, PIN – 208017

Mobile: +91-700-741-0690
Email: info@inertialelements.com
URL: www.inertialelements.com
www.gt-silicon.com

Revision History

Revision	Revision Date	Updates
1.0	10 August 2018	Initial version
1.1	13 August 2018	Removed typos Included point of differences between MIMU4844 & MIMU48XC and Features list
1.2	14 August 2018	Included examples of converting hex values to actual values
1.3	31 August 2018	Data format of the command "Osmium Output Data" is changed according to the new firmware which came into effect on 1st March, 2018.
1.4	4th January 2021	New command included (Command # 13 in Table1, and corresponding description). It is valid for MIMU4844 only. The modules shipped on or after 28th Dec, 2020, contain this command.
1.5	14th May 2021	Command # 13 (Selecting/Deselecting IMUs in Precision mode) in Table1 extended for MIMU48XC also. The modules shipped on or after 14th May 2021 contain this command.

Purpose & Scope

This document describes the data processing flow in **MIMU4844 and MIMU48XC**. It also describes communication protocol using which one can access and control the data and the processing at various stages, through an external application platform.

Refer following documents also for specification details:

1. Datasheet –ICM-20948
2. 32-bit AVR Microcontorller Specification

Refer the following document for Stepwise Data and Packet Acknowledgement:

1. Application Note

Introduction

The MIMU4844/48XC is Multi-IMU based inertial navigation module. MIMU4844/48XC, with on-board four 9-DOF IMUs, is targeted for carrying out research in motion sensing, robotics, IoT etc.

Due to on-board 32-bits floating point controller, device has a simplified dead reckoning interface (for foot-mounted application). An application platform receives a stream of displacement and heading changes, which it sums up to track the carrier. This is a significant simplification compared with handling and processing high-rate raw inertial measurements.

The MIMU48XC is a variant of massive inertial sensory array module MIMU4844, with a 10-pins connector for primarily allowing access to UART and SPI IOs of the micro-controller. In short, *MIMU48XC = MIMU4844 + 10-pins Connector*. MIMU48XC, when used with the extension board BMBT4444, is capable of storing data in micro SD card, operating on battery power and most importantly, wireless (Bluetooth) communication with the application platform.

MIMU4844/48XC is mainly targeted to be used as a precision wireless IMU for other non foot mounted applications such as robotics, VR/AR etc. In summary, presence of on-board 32-bits floating point controller enables on-board computing and hence simplified output data format, with significantly reduced data transfer rate.

This document describes the data processing flow in MIMU4844. It also describes communication protocol using which one can access and control the data and the processing at various stages, through an external application platform.

Highlighting Features:

- A Massive 32 IMUs array: Two 4x4 IMU arrays on each side of the board.
- Array of nine-axis IMU (Gyro + Accelero + Magnetometer).
- IMUs' placement and orientation to minimize systematic errors.
- Parallel communication with IMUs using 32 parallel s/w I2C buses.
- Sensor fusion and calibration compensation on-board.
- Sensors' sampling rate of 562.5 Hz
- Floating pt controller AT32UC3C with 512 Kb internal flash memory.
- USB 2.0 data interface; All sensors accessible through USB.

- JTAG programmable (needs dedicated JTAG cable).
- Power with USB; LED indicators; 49.3mm x 26.6mm.

MIMU48XC = MIMU4844 + 10-pins Connector (for allowing access to UART and SPI IOs of the micro-controller)

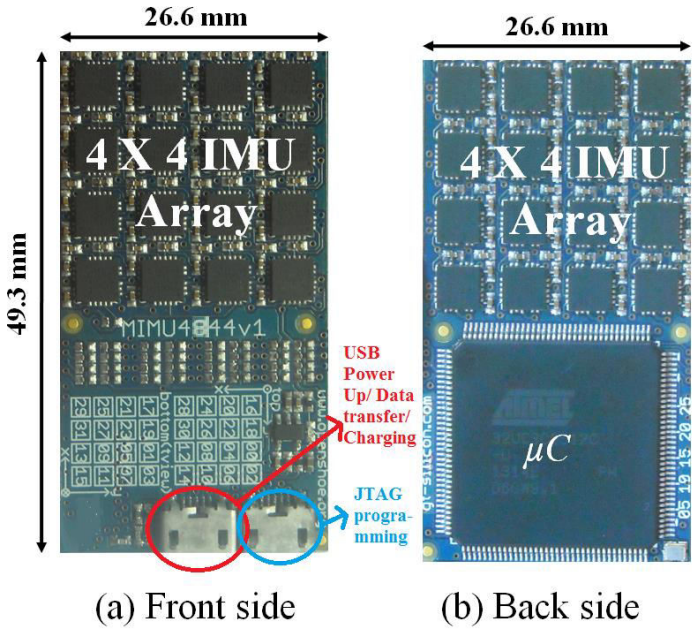

Fig 1.1: MIMU4844

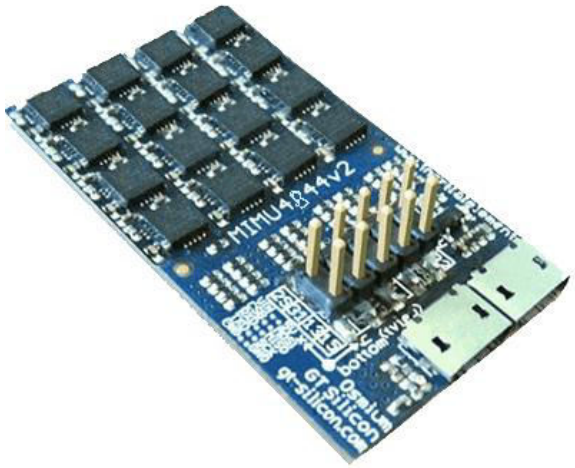

Fig 1.2: MIMU48XC

Physical dimensions:

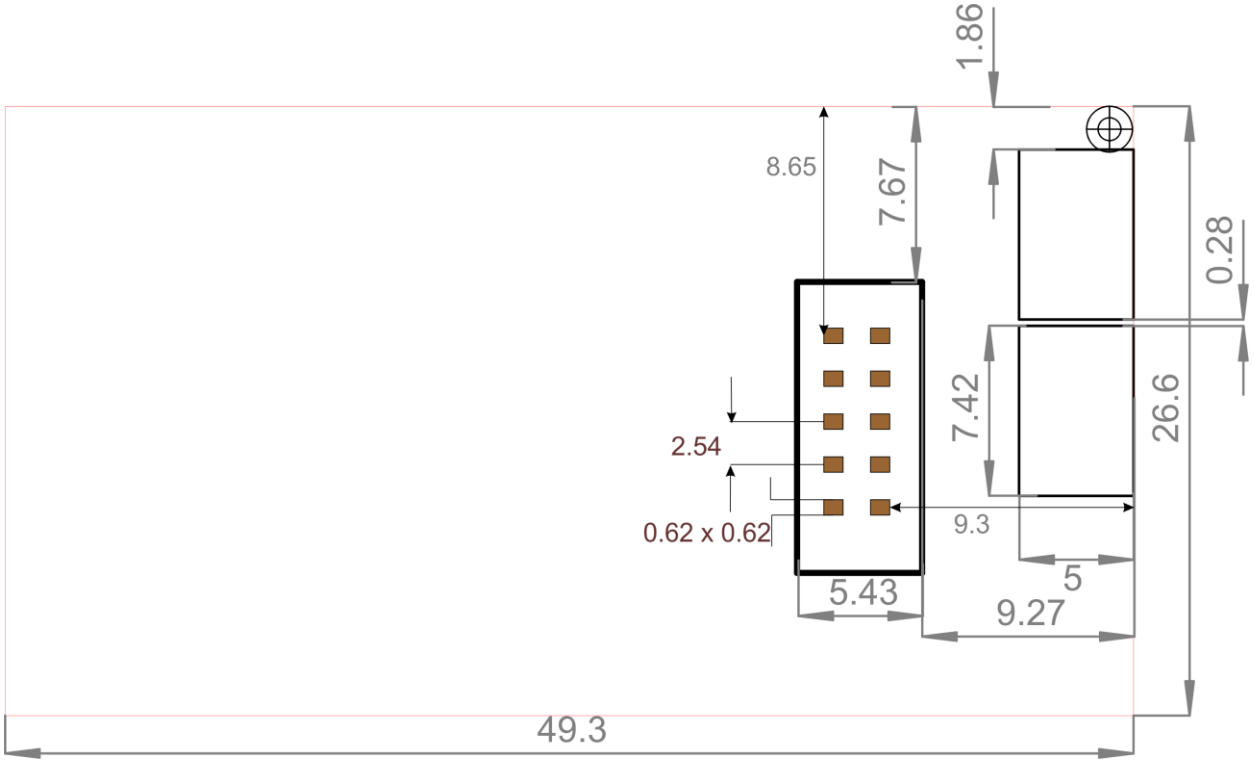

Fig 2: Top side of MIMU48XC. All dimensions in mm.

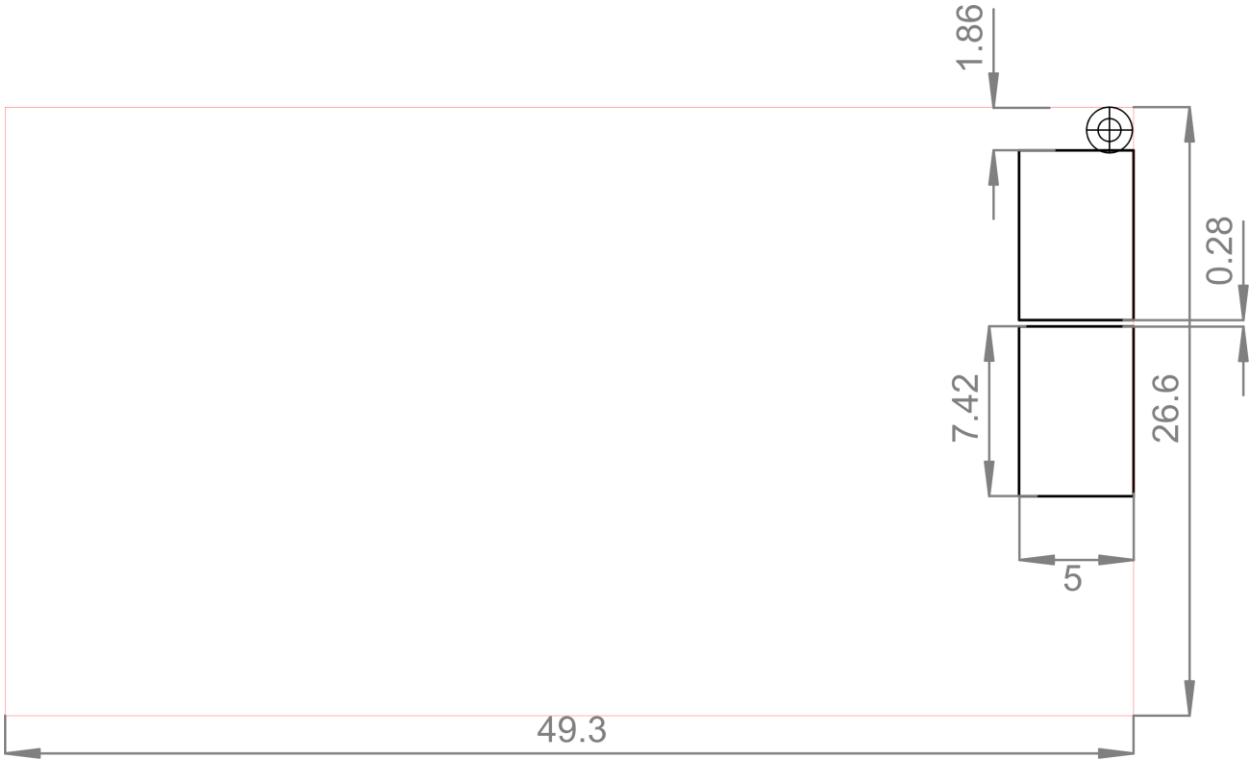

Fig 3: Top side of MIMU4844. All dimensions in mm.

IMUs' Numbering & Orientation

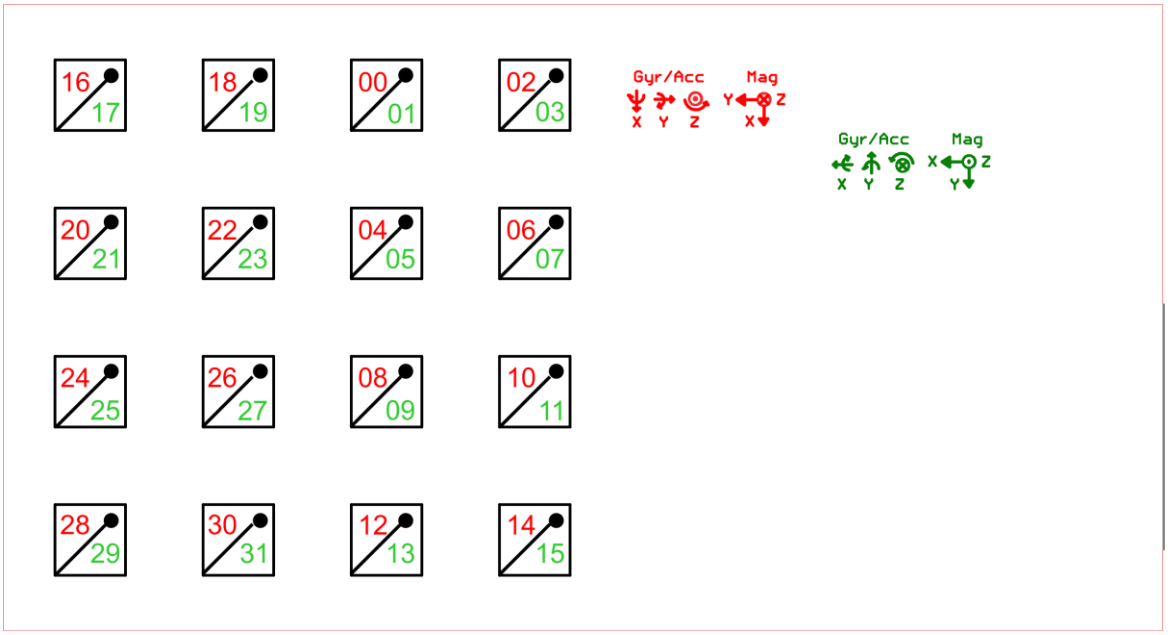

Fig 4: IMUs' numbering & orientation as appear from the top side of the board. IMUs on top and bottom are exactly mirrored. This means that the "dot" indication marks of the top and bottom IMUs are aligned. Red: Representing IMUs on top side of the board. Green: IMUs on bottom side of the board. This image is as appears from the top. Top: The side of the board which has USB connectors.

USB Driver Installation

One must install USB driver for data communication with PC (or any data acquisition system).

The on-board microcontroller Atmel AT32UC3C0512C offers USB 2.0 port. The USB connector on MIMU4844/48XC is directly connected with the USB pins of the controller.

The USB port of the microcontroller has been programmed in the firmware as CDC Virtual COM, so that it appears as a UART port (COM port) when connected with a PC. Hence installation of a corresponding USB driver on PC (or data acquisition system) is required, if the board has to communicate with the PC through its USB port.

(i) Download the Atmel USB driver file and its installation manual from https://www.inertialelements.com/resources.html.

(ii) Go to Computer management/Device Manager/Port and update the driver as instructed in the manual. *Though the installation process is demonstrated for Windows 10, it is same for the previous versions of Windows OS.*

For video demo, visit the support page from www.inertialelements.com and check the video with title "How to update USB Driver for MIMU22BL"

Data Flow

Sensors' raw data from all the 32 IMUs are read in parallel by the microcontroller through I2C buses realized in software. The sensors' data is calibration compensated and fused (averaged) to get single a_x, a_y, a_z, g_x, g_y and g_z. ZUPT aided inertial navigation is implemented, which generates displacement and heading change data.

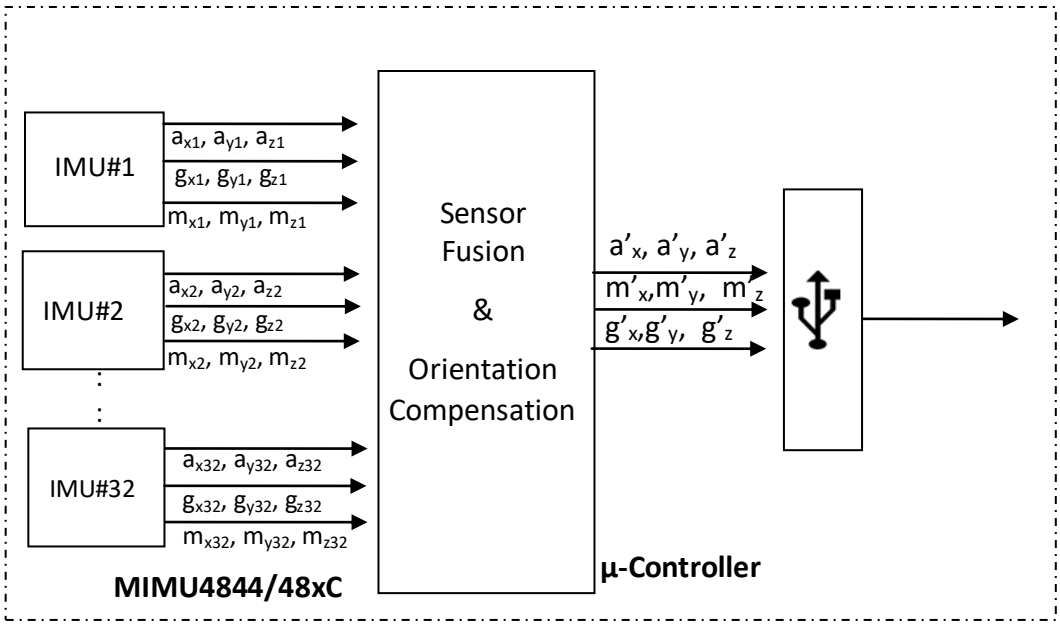

Figure 5 Illustration of data flow sequence in MIMU4844/48xC with ZUPT-aided inertial navigation

Communication Protocol

The interfacing application platform (Computer, Phone, Tab etc) sends a particular command (data request) to MIMU4844/48XC which serves the request by sending out required data to the application platform. Each command calls a particular state, which can send out variety of data depending upon command options.

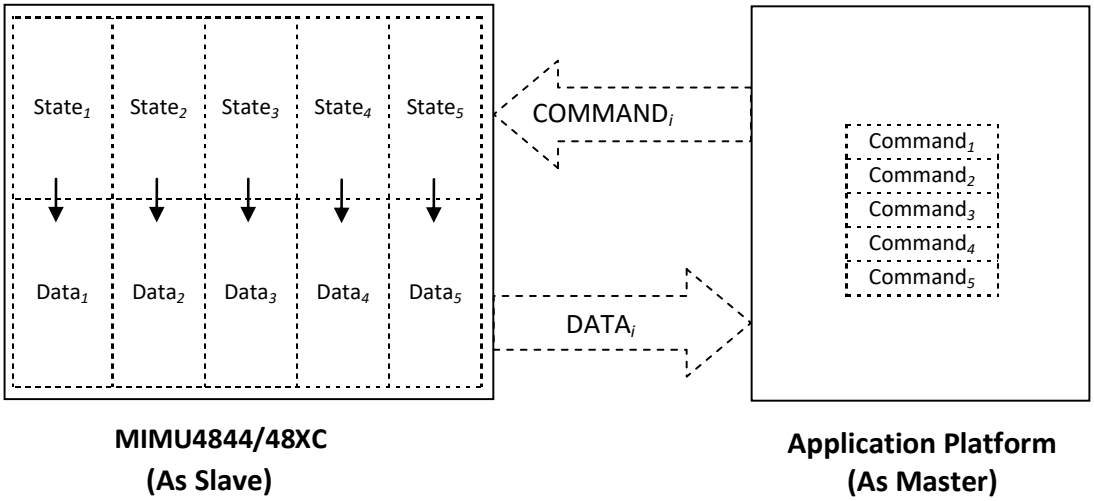

Figure 6 Data communication protocol between MIMU4844/48XC and the application platform

Command format = [command header,{payloads}, checksum*]

Checksum is of two bytes (cs1, cs2). Last two bytes in any command are checksum. Checksum is the modular sum of all the bytes in the payload in unsigned 2 byte integer format. *Refer Appendix III(1) for checksum computation.*

All the commands are represented in decimal format unless otherwise specified.

All the outputs from MIMU4844/48XC are in big – endian format. Those are represented in hex format in this document.

Response to any command

On giving any command to MIMU4844/48XC, there will be a command acknowledgement. The format of the command acknowledgement in response to any command is as follows:

Command Acknowledgement = [***0xA0*** CH cs1 cs2]

where CH -> command header i.e if the command is [0x40 0x01 0x00 0x41] so 0x40 is the command header so the command acknowledgement will be ***=>0xA0 0x40 0x00 0xE0.*** *Refer to Appendix III(2) for details.*

Table 1: Summary of some useful commands

Sl. No.	Command	Description	Syntax (in decimal)
1	Stepwise Dead Reckoning (PDR)	MIMU4844/48XC outputs displacement (dx, dy, dz) & change in orientation (dθ) w.r.t the previous detected step and entries of a 4x4 symmetric covariance matrix	[52 0 52] cs1 = 0 cs2 = 52
2	Processing off	MIMU4844/48XC (controller) switches off all the processing. It makes the processing sequence empty	[50 0 50] cs1 = 0 cs2 = 50
3	All output off	MIMU4844/48XC turns off output of all the states and stops transmitting data to the application platform	[34 0 34] cs1 = 0 cs2 = 34
4	Read inertial data	MIMU4844/48XC starts sampling onboardIMUs' (acceleros' and gyros' data)	[48 19 0 0 67] cs1 = 0 cs2 = 67
5	Output inertial data	MIMU4844/48XC transmitting the inertial sensors' raw data axi, ayi, azi, gxi, gyi, gzi of the selected IMUi, to the application platform.	[40 pl1 pl2 pl3 pl4 pl5 cs1 cs2] pl1, pl2, pl3, pl4 all together consists of 32bits. Each bit corresponds to particular IMU. Thus pl1-pl4 is used to select IMUs[$]. pl5 is the output mode*. cs1, cs2: checksum
6	Read Osmium Data	MIMU4844/48XC starts sampling onboardIMUs 'calibrated magnetometer and Inertial as well as pressure sensor data simultaneously	[48 22 0 0 70] cs1 = 0 cs2 = 70
7	Output Osmium Data	MIMU4844/48XC transmitting the inertial sensors' raw data axi, ayi, azi, gxi, gyi, gzi, magnetometer raw data mxi, myi, mzi and temperature data of the selected IMUsand Pressure and temperature data from Pressure sensor to the application platform.	[43 pl1 pl2 pl3 pl4pl5 cs1 cs2] pl1, pl2, pl3, pl4 all together consists of 32bits. Each bit corresponds to particular IMU. Thus pl1-pl4 is used to select IMUs[$]. pl5 is the output mode*. cs1, cs2: checksum
8	Read IMU temperature data	MIMU4844/48XC starts reading internal temperature of the onboard IMUs'	[53 0 53] cs1 = 0 cs2 = 53
9	Output IMU temperature data	MIMU4844/48XC starts	

		transmitting temperature of the selected onboard IMU(s), to the application platform.	[40 pl1 pl2 pl3 pl4 pl5 cs1 cs2] pl1, pl2, pl3, pl4 all together consists of 32bits. Each bit corresponds to particular IMU. Thus pl1-pl4 is used to select IMUs[$]. pl5 is the output mode*. cs1, cs2: checksum
10	High precision IMU (Normal IMU)	MIMU4844/48XC outputs g'x, g'y, g'z, a'x, a'y, a'z (ref Fig 5 and Fig 6) after calibration compensation and sensor fusion.	[64 pl1 cs1 cs2] pl1 = output mode* cs1, cs2: checksum
11	High precision IMU with bias estimation	MIMU4844/48XC outputs g'x, g'y, g'z, a'x, a'y, a'z after calibration compensation and sensor fusion along with the estimated bias.	[65 pl1 cs1 cs2] pl1 = output mode* cs1, cs2: checksum
12	High precision IMU with calibrated magnetometer # 4 data	MIMU4844/48XC outputs g'x, g'y, g'z, a'x, a'y, a'zafter calibration compensation and sensor fusion and calibrated magnetometer # 4	[49 22 16 0 0 0 0 0 0 0 87] [33 08 19 164 0 0 0 0 0 pl1 cs1 cs2] pl1 = output mode* cs1, cs2: checksum
13	High precision IMU with calibrated magnetometer with selection of IMUs (Only valid for selection of any combinations of 1, 2, 4, 8, 16 and 32 IMUs. Selecting any other combination results in invalid data.) [*It is introduced on 28th Dec 2020 for MIMU4844 and on 14th May 2021 for MIMU48XC.*]	MIMU4844/48XC outputs g'x, g'y, g'z, a'x, a'y, a'z after calibration compensation and sensor fusion and calibrated magnetometer of the lowest IMU# from different combination of selection of IMUs	[71 pl1 pl2 pl3 pl4 pl5 cs1 cs2] pl1, pl2, pl3, pl4 all together consists of 32bits. Each bit corresponds to particular IMU. Thus pl1-pl4 is used to select IMUs[$]. pl5 is the output mode*. cs1, cs2: checksum

***Output mode (MODE):** Its 1 byte output mode.

MODE[3:0] - Output rate divider$ (0x0 to 0x0f).

MODE[4] -1: Lossless transmission[##] of output data packet.

0: Not a lossless transmission of output data packet.*(Recommended for accessing sensors' data.)*

MODE[5] - This bit is meaningful only if Output Rate Divider[#] (RD) is set to 0.

1: MIMU4844/48XC generates only a single output data packet in response to a command.

0: MIMU4844/48XC does not generate any output data packet.

MODE[6] - Packet contains raw IMU data.

MODE[7] - Packet contains IMU temperature data.

[##]Lossless transmission: For lossless transmission, each data packet must be acknowledged by the application platform. Therefore, MIMU4844/48XC must receive ACK for every output data packet. In this mode, MIMU4844/48XC transmits next data packet, only on receiving acknowledgement for the

previous one. The data acknowledgement ("PKG ACK") packet is different from command acknowledgement (ACK received in response to command). *Sizes of PKG ACK and ACK (command) packets are 5 bytes and 4 bytes respectively. Refer Appendix III for details about acknowledgement packet.*

Table 2: Output mode Sample

Output mode (Binary Bits)	Rate divider MODE[3:0]	Lossless MODE[4]	Single time output MODE[5]	Acc + Gyro from each IMU MODE[6]	Temp from each IMU MODE[7]	Other Output*[1]	Decimal Equivalent
0000 0001 - 0000 1111	1-15	X	X	X	X	✓	1-15
0100 0001 – 0100 1111	1-15	X	X	✓	X	X	65-79
0101 0001 – 0101 1111	1-15	✓	X	✓	X	X	81-95
1001 0001 – 1001 1111	1-15	✓	X	X	✓	X	145-159
1000 0001- 1000 1111	1-15	X	X	X	✓	X	129-143
0110 0000	0	X	✓	✓	X	X	96
00010001- 00011111	1-15	✓	X	X	X	✓	17-31
00100000	0	x	✓	X	X	✓	32
00110000 (No use)	0	✓	✓	X	X	✓	48

*[1]: Other output includes all data output commands where output mode is used except "Output inertial data" and "Output IMU temperature data".

In the above table:

X: Feature disabled

✓: Feature enabled

[#]**Output Rate Divider (RD):** It constitutes of only one byte. It is used to manipulate the rate of output of the particular state asked in this command. If sensors' data read rate is 'f', then the output data rate of this particular state will be f/ (2^ (pl5-1)). The largest value of pl2 can be 15. Thus if pl5=1 (smallest value), then the particular state will be output for every data read from sensors.

In very simple words, RD decides the rate of data transmission from MIMU4844/48XC to the application platform. The maximum transmission rate is 562.5 Hz which is same as the inertial sensors' sampling frequency. Transmission rates for the RD upto 7 are given in the following table:

Output Rate Divider	Data Transmission Rate (Hz)
1	562.5
2	281.25
3	140.625
4	70.3125
5	35.156
6	17.578
7	8.789

If the output rate divider is 32 then there will be a single packet output.

[$]**Selection of IMUs**: 4 byte payloads pl1-pl4 consists of 32 bits where each bit corresponds to an IMU. For output of raw data from any combinations of IMUs set the bit corresponding to that IMU. Thus by setting any bit data from any combination of IMUs is possible. Refer Tables 3.1, 3.2 and 3.3 for better understanding.

Table 3.1: Selection of IMUs: Enabling sensors' data from all the IMUs

IMU No.	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Enabling all IMUs	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Hex	0xFF / 255								0xFF / 255								0xFF / 255								0xFF / 255							

Table 3.2: Selection of IMUs: Enabling sensors' data from IMU#0 to IMU#15

IMU No.	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Enabling IMU#0 to #15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Hex / Dec	0x00 / 0	0x00 / 0	0xFF / 255	0xFF / 255

Table 3.3: Selection of IMUs: Enabling sensors' data from IMU#16 to IMU#31

IMU No.	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Enabling IMU#16 to #31	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Hex / Dec	0xFF / 255								0xFF / 255								0xFF / 0								0xFF / 0							

Output packet format:

Output data packet is of the following format: **[Output header, payload, checksum]**

Each data packet contains

- 4 bytes output header which consists of packet header (1 byte), packet number (2 byte) and payload size (1 byte).*Refer to Appendix 3(3) for details.*
- payload – This is different from command to command
- 2 bytes checksum. *Refer to Appendix 3(1) for details.*

Refer Appendix I for detailed description of Commands and States.

Appendix I

Description of some important commands is presented in this section.

Note:

(i) Application platform receives an acknowledgement packet from MIMU4844/48XC in response to every command. Application platform acknowledges every data packet received from MIMU4844/48XC by sending an acknowledgement packet. The ACK in response to a command is different from the ACK in response to a data packet. *Refer Appendix III to know the acknowledge packet formats.*

(ii) MIMU4844 allows data transmission over USB only. Whereas, user makes choice between USB and Bluetooth for receiving data from MIMU48XC (along with the extension board BMBT4444). Choice of USB or Bluetooth depends upon the data transfer rate or the bandwidth requirement. Bluetooth can support limited bandwidth. Therefore it must be avoided for transmitting large and high speed data packets.

1. **State**: Stepwise Dead Reckoning (SWDR)
   Command: [52 0 52]

   MIMU4844/48XC sends out data packet for stepwise dead reckoning in response to this command.Visit the support page from www.inertialelements.com and check the "Application Note" under MIMU22BL Documents under "Resources" TAB for details about the command.

2. **State**: Processing Off
   Command: [50 0 50]

   MIMU4844/48XC terminates all the ongoing processing in response to this command. This is soft OFF button. There would not be any output to this command. On giving this command there will be only a command acknowledgement. This command is recommended as soon as the user finishes collecting data preceded by (3)i.e All Output Off command.

3. **State**: All Output Off
   Command: [34 0 34]

   If this command is passed to the system, then the transmission of data packets would stop. But there will be no change in the process going on inside, that all the functions would be running as

it is, just the thing is values would not be output. There would not be any output to this command. On giving this command there will be only a command acknowledgement. This command is also recommended as soon as the user finishes collecting data followed by (2) i.e. "Processing Off" command.

4. **State**: Read Inertial Data
   Command: [48 19 rd cs1 cs2]

   This command creates enabling conditions for the command Output Inertial Data, i.e. (4). Therefore this command must be run just before (5). There would not be any output to this command. On giving this command there will be only a command acknowledgement.

   rd: Output rate divider
   cs1, cs2: Two bytes checksum
   Example:
   Recommended command sequence for USB:
   [48 19 0 0 67]
   [40 0 0 0 15 65 0 120] (o/p data rate:1 = 562.5 Hz) (Details in (5))

   Refer Appendix III for Checksum.

5. **State**: Output Inertial Data
   Command: [40 pl1 pl2 pl3 pl4 pl5 cs1, cs2]

   MIMU4844/48XC outputs IMU sensors' readings in response to this command.

   Here pl1, pl2, pl3 and pl4 constitute one byte each. These are used for selecting IMUs. Consider the pl1, pl2, pl3 and pl4 all together constitute 4 bytes i.e. 32 bits, where each bit corresponds to one of the 32 IMUs.

   Consider if we want to select the first four IMUs, set four LSBs to 1 i.e. assign value of 15 (binary: 00001111) to pl4 and all the others be 0. Thus [40 0 0 0 15 1 0 56] is a sample command selecting the first four IMUs for getting the raw inertial data. Refer the Table # 3 to have a better understanding on selection of IMUs.

pl5: Output mode byte. Here pl5 is 64 + the actual output rate divider (1, 2, 3… etc).
cs1, cs2: Two bytes checksum

Here the output consists of [a_{xi}, a_{yi}, a_{zi}, g_{xi}, g_{yi}, g_{zi}] for each IMU selected, as in Fig 5. As each of the values corresponds to 2 bytes integer, thus the output is of 12 bytes for each IMU.

Note: (3) i.e. command Read Inertial Data must be run before this command.

Example 1:*Set of commands to obtain acc and gyro data from IMU#0 to IMU#3.*

[48 19 0 0 67]
[40 0 0 0 15 65 0 120] (o/p data rate:1 = 562.5 Hz)

The above command is recommended by USB. Bluetooth may not support the required data transmission rate.

Here are the example data packets containing 4 accelerometers' and 4 gyroscopes' data, obtained in response to the above set of commands (through USB): Two sample data packets are shown below:

0xAA 0x00 0x01 0x34 0x21 0xC4 0x48 0xF7 0x00 0x6E 0x00 0x54 0x07 0x57 0xFF 0xE4 0x00 0x05 0x00 0x17 0xFF 0x98 0xFF 0x81 0xF7 0x6A 0xFF 0xD2 0x00 0x13 0x00 0x11 0x00 0x7E 0x00 0x57 0x07 0xD9 0xFF 0xF6 0x00 0x0B 0xFF 0xEF 0xFF 0x9A 0xFF 0x8C 0xF7 0x60 0xFF 0xF6 0xFF 0xF0 0xFF 0xFC 0x1C 0x8C

0xAA 0x00 0x02 0x34 0x21 0xC5 0x35 0xEC 0x00 0x6E 0x00 0x54 0x07 0x57 0xFF 0xE2 0x00 0x04 0x00 0x16 0xFF 0x90 0xFF 0x7F 0xF7 0x68 0xFF 0xD4 0x00 0x13 0x00 0x11 0x00 0x68 0x00 0x56 0x07 0xD0 0xFF 0xF5 0x00 0x0A 0xFF 0xF0 0xFF 0x8E 0xFF 0x91 0xF7 0x54 0xFF 0xF2 0xFF 0xF2 0xFF 0xFE 0x1C 0x2E

***Total packet size**: 58 bytes (4 bytes Output Header + 52 bytes Payload + 2 bytes Checksum)*

First 4 bytes (Header): *0xAA 0x00 0x01 0x34 (0xAA – packet header; 0x00 0x01 - Data packet number; 0x34 – Payload size). Refer Appendix III to know more about Header.*

Next 52 bytes (Payload)*: First 4 bytes (unsigned integer) of payload are time stamp*; 2 bytes integer each for ax1, ay1, az1, gx1, gy1, gz1, ax2, ay2, az2, gx2, gy2, gz2, ax3, ay3, az3, gx3, gy3, gz3, ax4, ay4, az4, gx4, gy4 and gz4.*

Last 2 bytes (Checksum)*: 0x1C 0x8C. Refer Appendix III for Checksum.*

**Timestamp is the integer value of a 32bit counter which is running continuously on system clock since power-on reset.*

Example 2: Set of commands to obtain acc and gyro data from IMU#0 to IMU#31.

[48 19 0 0 67]
[40 255 255 255 255 65 04 101] (o/p data rate:1 = 562.5 Hz)

The above command is recommended by USB. Bluetooth may not support the required data transmission rate.

0xAA 0x00 0x01 0x84 0x90 0x45 0x86 0xFE 0x00 0x10 0xFF 0xCF 0x08 0x15 0xFF 0xF9 0x00 0x09 0x00 0x09 0xFF 0xCE 0xFF 0xAE 0xF7 0xF2 0xFF 0xEA 0x00 0x0A 0xFF 0xFB 0xFF 0xF2 0xFF 0xA1 0x07 0xEF 0x00 0x0F 0x00 0x01 0xFF 0xF2 0xFF 0xCB 0xFF 0xA7 0xF7 0xD2 0x00 0x0C 0xFF 0xF9 0xFF 0xFE 0xFF 0xFC 0xFF 0xD7 0x07 0xFA 0xFF 0xFD 0xFF 0xF8 0x00 0x03 0xFF 0xD2 0xFF 0x80 0xF7 0xDA 0xFF 0xE5 0x00 0x0B 0xFF 0xF7 0xFF 0xEF 0xFF 0xB2 0x08 0x14 0x00 0x1E 0xFF 0xFB 0x00 0x10 0xFF 0xE1 0xFF 0x84 0xF7 0xF2 0xFF 0xF7 0xFF 0xF9 0xFF 0xFF0xFF0xF9 0xFF 0xC9 0x08 0x22 0xFF 0xE5 0xFF 0xF0 0xFF 0xFB 0xFF 0xD6 0xFF 0xAC 0xF7 0xC3 0x00 0x02 0x00 0x08 0x00 0x03 0xFF 0xF7 0xFF 0xEB 0x08 0x0A 0xFF 0xFC 0xFF 0xFB 0xFF 0xFE 0xFF 0xFA 0xFF 0xCB 0xF7 0xD3 0xFF 0xEF 0xFF 0xED 0x00 0x05 0xFF 0xF5 0xFF 0xB0 0x08 0x27 0x00 0x08 0xFF 0xFA 0x00 0x03 0xFF 0xE7 0xFF 0x9C 0xF7 0xE2 0xFF 0xF1 0x00 0x03 0x00 0x0A 0xFF 0xE8 0xFF 0xF9 0x08 0x3B 0x00 0x09 0x00 0x02 0xFF 0xFA 0xFF 0xE5 0xFF 0xA8 0xF7 0xC5 0x00 0x08 0xFF 0xFF0xFF0xEB 0xFF 0xEC 0xFF 0xE5 0x08 0x18 0x00 0x000xFF 0xEF 0xFF 0xFD 0xFF 0xD3 0xFF 0xAE 0xF7 0xBF 0x00 0x06 0x00 0x09 0xFF 0xF9 0xFF 0xF8 0xFF 0xAB 0x08 0x0D 0xFF 0xE9 0xFF 0xF8 0x00 0x0C 0xFF 0xD2 0xFF 0x8F 0xF7 0xE4 0xFF 0xE5 0x00 0x03 0xFF 0xEE 0xFF 0xEF 0xFF 0xC8 0x08 0x2A 0x00 0x0B 0xFF 0xFE 0xFF 0xFD 0xFF 0xDC

0xFF 0xC9 0xF7 0xD8 0x00 0x12 0xFF 0xFF0x00 0x07 0xFF 0xFF0xFF0xD2 0x07 0xFA 0x00 0x13 0xFF 0xFF0x00 0x0B 0xFF 0xDC 0xFF 0xB0 0xF7 0xB6 0xFF 0xFA 0x00 0x07 0xFF 0xFF0xFF0xFE 0xFF 0xE1 0x08 0x0E 0x00 0x06 0x00 0x03 0x00 0x04 0xFF 0xEF 0xFF 0xD2 0xF7 0xB9 0xFF 0xE8 0xFF 0xF5 0xFF 0xF3 0xFF 0xFC 0xFF 0xBE 0x08 0x0C 0xFF 0xDE 0xFF 0xF3 0x00 0x000xFF 0xD4 0xFF 0xA1 0xF7 0xB0 0xFF 0xE6 0x00 0x0F 0x00 0x05 0xFF 0xF8 0xFF 0xF5 0x07 0xFF 0xFF0xF6 0x00 0x03 0x00 0x03 0xFF 0xC9 0xFF 0x9B 0xF7 0xAB 0x00 0x1D 0xFF 0xF3 0x00 0x0A 0xFF 0xF0 0xFF 0xEC 0x07 0xF6 0x00 0x10 0x00 0x08 0xFF 0xFC 0x00 0x10 0xFF 0x7C 0xF7 0x98 0xFF 0xFC 0x00 0x0B 0x00 0x1C 0xF9 0xD2

***Total packet size**: 394 bytes (4 bytes Output Header + 388bytes Payload + 2 bytes Checksum)*
***First 4 bytes (Header)**: 0xAA 0x00 0x 01 0x84 (0xAA – packet header; 0x00 0x01 - Data packet number; 0x84 – Payload size). Refer Appendix III(3) to know more about output header.*
***Next 388 bytes (Payload)**: First 4 bytes (unsigned integer) of payload are time stamp; 2 bytes integer each for ax1, ay1, az1, gx1, gy1, gz1, ax2, ay2, az2, gx2, gy2, gz2, ax3, ay3, az3, gx3, gy3, gz3, ax4, ay4, az4, gx4, gy4 and gz4 and so on.*
***Last 2 bytes (Checksum)**: 0xF9 0xD2. Refer Appendix III for Checksum*

How to obtain output of sensor in decimal:

To get the actual value of acceleration (from accelerometer) and angular rotation speed (from gyroscope) use the scale (multiplication factor) as given in the following table:

Data	Data type	Scale (Multiplication factor)	Unit
Acceleration	2 bytes integer	(1/2048.0)*gvalue [gvalue is 9.79 m/s^2]	meter per second square (m/s^2)
Angular rotation speed	2 bytes integer	1/16.4	degrees per second

Time stamp (5th to 8th byte) - 0x90 0x45 0x86 0xFE (4 bytes unsigned int.) =>2420475646, multiply with scale factor =>2420475646*(1/16e+6) => 151.2797 sec
ax1 (9th to 10th byte) - 0x00 0x10(2 bytes integer) => 16, multiply with scale factor => 16*(1/2048)+9.79 => 0.0765 m/s^2
gx1 (15th to 16th byte) - 0xFF 0xF9 (2 bytes integer) => -7, multiply with scale factor => -7*(1/16.4) => 0.4268 degree/sec

6. **State**: Read IMUs' Temperature

Command: [53 0 53]

This command creates enabling conditions for the command Output IMU Temperature Data, i.e. (7). Therefore this command must be run just before (7). There would not be any output to this command. Application platform receives only acknowledgement packet in response to this command.

7. **State**: Output IMU Temperature Data
   Command: [40 pl1 pl2 pl3 pl4 pl5 cs1 cs2]

   MIMU4844/48XC outputs temperature of the selected IMUs in response to this command. The output data packet consists of 2 bytes containing the value of internal temperature, for every selected IMU.

   pl1, pl2, pl3 and pl4 are same as in the command for state OUTPUT_IMU_RD. pl5 is 128 + the actual output rate divider (1, 2, 3…etc).

   cs1, cs2: Two bytes checksum

   Note: (6) i.e. command Read IMU's Temperature must be run before this command.

   Example:
   [53 0 53] (Enables reading IMUs' internal temperature)
   [40 0 0 0 15 129 0 184] (o/p data rate:1 = 562.5 Hz)

   Following is the example data packet, in response to the above set of commands, containing internal temperature information of just first 4 IMUs:

   0xAA 0xD0 0x28 0x0C 0x00 0x57 0x00 0x19 0xBF 0x16 0x00 0x1B 0x0F 0x2E 0x3F 0x9A 0x04 0x24

   ***Total packet size:** 18 bytes (4 bytes Output Header + 12 bytes Payload + 2 bytes Checksum)*
   ***First 4 bytes (Header)**: 0xAA 0xD0 0x28 0x0C (0xAA – packet header; 0xD0 0x28 - Data packet number; 0x0C - Payload size). Refer Appendix III(3) to know more about output header.*

***Next 12 bytes (Payload)**: First 4 bytes (unsigned integer) of payload are time stamp; 2 bytes integer each for temperatures from IMU1, IMU2, IMU3 and IMU4.*
***Last 2 bytes (Checksum)**: 0x04 0x24. Refer Appendix III for Checksum.*

How to obtain output of sensor in decimal: Following table contains information about the data received:

Data	Data type	Scale (Multiplication factor)	Addition factor	Unit
Temperature	2 bytes integer	1/340	35	Celsius

Time stamp (5th to 8th byte) - 0x00 0x57 0x00 0x19 (4 bytes unsigned int.) =>5701657, multiply with scale factor =>5701657*(1/16e+6) => 0.3563 sec
Temp from IMU 1 (9th to 10th byte) - 0xBF 0x16 (2 bytes integer) => -16618, multiply with scale factor => -16618*(1/340)+35 => 83.8765 ^{o}C

8. **State**: Read Osmium Data
   Command: [48 22 0 cs1 cs2]

   This command is used to initiate sampling and calibration of the onboard IMU's magnetometer, sampling of inertial sensors' data and sampling of the onboard pressure sensor data by MIMU4844/48XC simultaneously. There would not be any output to this command. On giving this command there will be only a command acknowledgement.

   rd: Output rate divider
   cs1, cs2: Two bytes checksum

   Refer Appendix III for acknowledge packet and Checksum

9. **State**: Output Osmium Data
   Command: [43 pl1 pl2 pl3 pl4 rd cs1 cs2]

   In response to the above command, MIMU4844/48XC transmits the sensors' raw data axi, ayi, azi, gxi, gyi, gzi, magnetometer raw data mxi, myi, mzi and raw temperature data of the selected IMUs.

pl1 pl2 pl3 pl4 – These payloads is for selection of IMUs.

rd- Output rate divider

cs1 cs2 - Check Sum

Example: *Command to obtain data packets containing sensors' (acc+gyro+mag+temp) data from IMU#0 to IMU#3.*

[43 00000015050063]

0xAA 0x01 0x76 0x78 0xB3 0xB0 0x1C 0xB2 0xFF 0x9A 0x00 0x4E 0x07 0x72 0xFF 0xEC 0xFF 0xEF 0x00 0x06 0xFF 0x8E 0xFF 0xFF0xF7 0x99 0xFF 0xFF0xFF0xF9 0xFF 0xF4 0x00 0x08 0x00 0x46 0x08 0x11 0x00 0x37 0xFF 0xF9 0x00 0x05 0xFF 0xF8 0x00 0x1E0xF7 0xCF 0xFF 0xCC 0x00 0x07 0xFF 0xF5 0x26 0x80 0x26 0xC0 0x26 0x20 0x25 0xE0 0x C0 0xFA 0xCC 0xCC0x3E 0xB6 0x66 0x60 0x3F 0x84 0x4C 0xCE 0x41 0x1B 0x6B 0x34 0xBF 0x38 0xCC 0xCC 0xC0 0xB1 0x99 0x9A 0xC0 0xCB 0x2C 0xCE 0xC1 0x4D 0xE0 0x00 0x41 0x89 0x58 0x00 0x42 0x0C 0x1C 0x00 0x41 0x89 0xB3 0x33 0xC1 0xDD 0x40 0x00 0x3F 0x77 0x63 0x33 0xBF 0x89 0xAC 0x00 0xBF 0x96 0x28 0xCC 0x3E 0x4D

1st 4 bytes: *0xAA 0x01 0x76 0x78: Output header;-0xAA-Packet Header; 0x01 0x76- Data packet number, 78-Payload size.*

Next 4 bytes: *0xB3 0xB0 0x1C 0xB2: Time stamp in unsigned integer format*

Next 48 bytes: *2 bytes each for ax1, ay1, az1, gx1, gy1, gz1,ax2, ay2, az2, gx2, gy2 and gz2 so on (in 2 bytes integer format)*

Next 8 bytes: *2bytes each for temp1, temp2, temp3 and temp4 (in 2 byte integer format)*

Next 48 bytes: *4 bytes float (IEEE 754 floating point representation) each for mx1, my1, mz1, mx2, my2, mz2, mx3, my3 and so on.*

Next 12 bytes: *4 bytes float (IEEE 754 floating point representation) each for fused magnetometer data of mx, my and mz.*

Last 2 bytes: *0x3E 0x4D - Check sum. Refer Appendix III for checksum.*

How to obtain output of sensor in decimal:

Following table contains multiplication and addition factors for data received.

Data	Data type	Scale (Multiplication factor)	Addition factor	Unit
Acceleration	2 bytes integer	(1/2048.0)*gvalue [gvalue is 9.79 m/s^2]	0.0	meter per second square (m/s^2)
Angular rotation speed	2 bytes integer	1/16.4	0.0	degrees per second
Magnetic field strength	4 bytes float (IEEE 754 floating point representation)	0.3	0.0	micro Tesla
Temperature	2 bytes integer	1/340	35	Celsius

Time stamp (5th to 8th byte) - 0xB3 0xB0 0x1C 0xB2 (4 byte unsigned int.) =>3014663346, multiply with scale factor =>3014663346*(1/16e+6) => 188.4165 sec

ax1 (9th to 10th byte) - 0xFF 0x9A (2 bytes integer) => -102, multiply with scale factor => -102*(1/2048.0)*9.79 => -0.4876 m/sec^2

gx1 (15th to 16th byte) - 0xFF 0xEC (2 bytes integer) =>-20, multiply with scale factor => -20*(1/16.4) => -1.2195 degree/sec

temp1 (57th to 58th byte) - 0x26 0x80 (2 bytes integer) =>9856, multiply with scale factor =>9856*(1/340)+35 => 63.9882 °C

mx1 (65th to 68th byte) - 0xC0 0xFA 0xCC 0xCC (4 bytes float) =>-7.83749962, multiply with scale factor =>-7.83749962*0.3 => -2.3512 mTesla

10. **State**: High precision IMU (Normal IMU)

    Command: [64 pl1 cs1 cs2]

    pl1 = output rate divider

    cs1, cs2: checksum

    With this command, MIMU4844/48XC can be used as a single high precision IMU. The modules are capable of giving calibration compensated and fused data from the IMU array. This means the module can be used as a single precision IMU which outputs 3-axis acceleration and 3-axis

gyroscope data. The device outputs g'_x, g'_y, g'_z, a'_x, a'_y, a'_z (ref Fig 5 and Fig 6) – 4 bytes each, IEEE 754 float type.

Below is the sample data from MIMU4844/48XC lying tilted on a table. The data was collected at data rate 562.5Hz.

Here is single sample packet for the precision IMU command:

0xAA 0x17 0x6C0x1C 0xC3 0xE6 0x7A 0x98 0x40 74 0x22 0x11 0x40 0x7B 0x3D 0x0F 0x41 0x03 0x7D 0x75 0x3A 0x16 0xA9 0x04 0xBC 0x38 0xC7 0x5C 0x3B 0x44 0x67 0x1A 0x0B 0x3C

***1st 4 bytes**: 0xAA 0x17 0x6C 0x1C: Output header;-0xAA-Packet Header; 0x17 0x6C - Data packet number, 0x1C-Payload size. Refer Appendix 3(3) to know more about output header.*
***Next 4 bytes**: 0xC3 0xE6 0x7A 0x98: Time stamp in unsigned integer format*
***Next 24 bytes**: 4 bytes each for ax, ay, az, gx, gy, gz, (IEEE 754 floating point representation)*
***Last 2 bytes**: 0x0B 0x3C Check sum. Refer Appendix III for Checksum.*

How to obtain output of sensor in decimal:
Following table contains data type and units off sensors' data.

Data	Data Type	Unit
Acceleration	4 byte float (IEEE 754 floating point representation)	meter per second square (m/s^2)
Angular rotation speed	4 byte float (IEEE 754 floating point representation)	radian per second

Time stamp (5th to 8th)- 0xC3 0xE6 0x7A 0x98 (4 byte unsigned int.) =>3286661784, multiply with scale factor =>3286661784*(1/16e+6) => 205.4164 sec
ax (9th to 12th) - 0x40 0x74 0x22 0x11(4 byte float) =>3.81457925, multiply with scale factor =>3.81457925*1 => 3.8146 m/s^2
gx (21th to 24th) - 0x3A 0x16 0xA9 0x04 (4 byte float) =>0.0005747231, multiply with scale factor =>0.0005747231*1 => 0.0006 radian/sec

11. **State**: High precision IMU with gyro bias estimation
    Command: [65 pl1 cs1 cs2]

pl1 = output rate divider

cs1, cs2: checksum

This command gives the combined fused precision IMU data after calibration compensation and adding the estimated bias. The data packet output format will be same as the "High precision IMU" data except the estimated gyro bias is added to the fused gyro values with each data set.

Here is single sample packet for the precision IMU command:

0xAA 0x17 0x6D 0x1C 0xC3 0xEA 0x1A 0x5A 0x40 0x72 0x2A 0x66 0x40 0x74 0x6B 0x15 0x41 0x02 0x00 0x56 0xBB 0x81 0x90 0xAE 0xBB 0xC4 0x02 0x4C 0x3C 0x15 0x68 0x7D 0x0B 0xF7

***1st 4 bytes**: 0xAA 0x17 0x6D 0x1C: Output header;-0xAA-Packet Header; 0x17 0x6D - Data packet number, 0x1D-Payload size.*

***Next 4 bytes**: 0xC3 0xEA 0x1A 0x5A: Time stamp in unsigned integer format*

***Next 24 bytes**: 4 bytes each for ax, ay, az, gx, gy, gz, (IEEE 754 floating point representation)*

***Last 2 bytes**: 0x0B 0xF7 Checksum. Refer Appendix III for Checksum.*

How to obtain output of sensor in decimal:

Following table contains data type and units off sensors' data.

Data	Data Type	Unit
Acceleration	4 byte float (IEEE 754 floating point representation)	meter per second square (m/s^2)
Angular rotation speed	4 byte float (IEEE 754 floating point representation)	radian per second

Time stamp (5th to 8th) - 0xC3 0xEA 0x1A 0x5A (4 byte unsigned int.) =>3286899290, multiply with scale factor =>3286899290*(1/16e+6) => 205.4312 sec

ax (9th to 12th) - 0x40 0x72 0x2A 0x66 (4 byte float) =>3.7838378, multiply with scale factor =>3.7838378*1 => 3.7838 m/s^2

gx (21th to 24th) - 0xBB 0x81 0x90 0xAE (4 byte float) =>-0.00395401474, multiply with scale factor =>-0.00395401474*1 => -0.0040 radian/sec

12. High precision IMU with calibrated magnetometer # 4 data

Command1: [49 22 16 0 0 0 0 0 0 0 87] (Enabling Command)
Command 2: [33 0819 164 0 0 0 0 0 pl1 cs1 cs2] (Output Command)

pl1: output rate divider
cs1, cs2: checksum bytes

With this command, MIMU4844/48XC can be used as a single high precision IMU. The modules are capable of giving calibration compensated and fused data from the IMU array. This means the module can be used as a single precision IMU which outputs 3-axis acceleration and 3-axis gyroscope data and 3 axis magnetometer data.
Here is an example output where the command is send through USB.

Example:

[49 22 16 0 0 0 0 0 0 0 87] (Enabling Command)
[33 08 19 164 0 0 0 0 0 05 0 229]

Following is one of the output packets:

0xAA 0x00 0xDE 0x28 0x3B 0xB9 0x11 0xAE 0xC0 0x5D 0x71 0x54 0x40 0xA0 0x27 0x3B 0x41 0x07 0xC3 0x1E 0x3E 0x9B 0xB6 0xC4 0x3F 0x5F 0x9D 0x6A 0xBF 0x02 0x6D 0x38 0x42 0x7E 0x000x 00 0xC2 0xC9 0x00 0x00 0xC30x07 0x00 0x00 0x10 0x23

***1st 4 bytes**: 0xAA 0x00 0xDE 0x28: Header*
0xAA-Packet Header
0x00 0xDE- Data packet number
0x28- Payload size. Refer Appendix III for Header.
***Next 4 bytes**: 0x3B 0xB9 0x11 0xAE: Time stamp in unsigned integer format*
***Next 24 bytes**: 4 bytes each for ax, ay, az, gx, gy, gz, (IEEE 754 floating point representation)*
***Next 12 bytes**: 4 bytes float (IEEE 754 floating point representation) each for mx4, my4, mz4.*
***Last 2 bytes**: 0x10 0x23 - Checksum. Refer Appendix III for Checksum.*

How to obtain output of sensor in decimal:
Following table contains data type and units off sensors' data.

Data	Data type	Scale (Multiplication factor)	Unit

Acceleration	4 bytes float (IEEE 754 floating point representation)	1	meter per second square (m/s^2)
Angular rotation speed	4 bytes float (IEEE 754 floating point representation)	1	radians per second
Magnetic field strength	4 bytes float (IEEE 754 floating point representation)	0.3	micro Tesla

Time stamp (5thto 8th) - 0x3B 0xB9 0x11 0xAE (4 byte unsigned integer) => 1001984430, multiply with scale factor => 1001984430*(1/16e+6) => 62.6240 sec

ax (9th to 12th) - 0xC0 0x5D 0x71 0x54 (4 byte float) => -3.460042, multiply with scale factor => -3.460042*1 => -3.4600 m/s^2

gx (21th to 24th) - 0x3E 0x9B 0xB6 0xC4 (4 byte float) => 0.304128766, multiply with scale factor => 0.304128766*1 => 0.3041 radian/sec

mx4 (33th to 36th) - 0x42 0x7E 0x00 0x00 (4 byte float) => 63.5, multiply with scale factor => 63.5*0.3 => 19.05 mTesla

13. High precision IMU with calibrated magnetometer data from different selection of IMUs [This command is applicable for MIMU4844 only.]

    Command: [71 pl1 pl2 pl3 pl4 pl5 cs1 cs2]

    Here pl1, pl2, pl3 and pl4 constitute one byte each. These are used for selecting IMUs. Consider the pl1, pl2, pl3 and pl4 all together constitute 4 bytes i.e. 32 bits, where each bit corresponds to each of the 32 IMUs, so if any bit at particular location is =1 then that IMU is activated , or if it is 0 then that IMU is off.; cs1, cs2: checksum bytes

    With this command, MIMU4844 can be used as a single high precision IMU. The modules are capable of giving calibration compensated and fused data from the IMU array. This means the module can be used as a single precision IMU which outputs 3-axis acceleration and 3-axis gyroscope data and calibrated 3 axis magnetometer data of the lowest IMU# selected.
    Here is an example output where the command is send through USB. **Remember only 1,2,4,8,16 and 32 selection of IMUs are valid. Any other selection will give wrong output.**

Example:

[71 0 0 0 3 1 0 75]

Here pl4 =3 which is 11b which indicate 2 IMUs are activated IMU#0 and IMU#1.
Following is one of the output packets:

0xAA 0x00 0xDE 0x28 0x3B 0xB9 0x11 0xAE 0xC0 0x5D 0x71 0x54 0x40 0xA0 0x27 0x3B 0x41 0x07 0xC3 0x1E 0x3E 0x9B 0xB6 0xC4 0x3F 0x5F 0x9D 0x6A 0xBF 0x02 0x6D 0x38 0x42 0x7E 0x000x 00 0xC2 0xC9 0x00 0x00 0xC30x07 0x00 0x00 0x10 0x23

***1st 4 bytes**: 0xAA 0x00 0xDE 0x28: Header*

0xAA-Packet Header

0x00 0xDE- Data packet number

0x28- Payload size. Refer Appendix III for Header.

***Next 4 bytes**: 0x3B 0xB9 0x11 0xAE: Time stamp in unsigned integer format*

***Next 24 bytes**: 4 bytes each for ax, ay, az, gx, gy, gz, (IEEE 754 floating point representation)*

***Next 12 bytes**: 4 bytes float (IEEE 754 floating point representation) each for mx4, my4, mz4 of IMU#0.*

***Last 2 bytes**: 0x10 0x23 - Checksum. Refer Appendix III for Checksum.*

How to obtain output of sensor in decimal:

Following table contains data type and units off sensors' data.

Data	Data type	Scale (Multiplication factor)	Unit
Acceleration	4 bytes float (IEEE 754 floating point representation)	1	meter per second square (m/s^2)
Angular rotation speed	4 bytes float (IEEE 754 floating point representation)	1	radians per second
Magnetic field strength	4 bytes float (IEEE 754 floating point representation)	0.3	micro Tesla

Time stamp (5thto 8th) - 0x3B 0xB9 0x11 0xAE (4 byte unsigned integer) => 1001984430, multiply with scale factor => 1001984430*(1/16e+6) => 62.6240 sec

ax (9th to 12th) - 0xC0 0x5D 0x71 0x54 (4 byte float) => -3.460042, multiply with scale factor => -3.460042*1 => -3.4600 m/s^2

gx (21th to 24th) - 0x3E 0x9B 0xB6 0xC4 (4 byte float) => 0.304128766, multiply with scale factor => 0.304128766*1 => 0.3041 radian/sec

mx4 (33th to 36th) - 0x42 0x7E 0x00 0x00 (4 byte float) => 63.5, multiply with scale factor => 63.5*0.3 => 19.05 mTesla

Few other combinations and their output:

Example 1: Command =[71 0 0 255 255 1 2 70] -> Here pl3=255(11111111b) & pl4 =255(11111111b) so 16 IMUs are activated IMU#0 to IMU#15. The output with calibrated and fused inertial data of IMU#0 to IMU#15 and calibrated compensated magnetometer data of the IMU#0.

Example2: Command =[71 0 255 255 0 1 2 70] -> Here pl2=255(11111111b) & pl3 =255(11111111b) so 16 IMUs are activated IMU#8 to IMU#23. The output with calibrated and fused inertial data of IMU#8 to IMU#23 and calibrated compensated magnetometer data of the IMU#8.

Example3: Command =[71 1 3 1 0 1 0 77] -> Here pl1=1(1b) , pl2 =3(11b) & pl3 = 1(1b) so 4 IMUs are activated IMU#24, IMU#16,IMU#17 & IMU#8. The output with calibrated and fused inertial data of IMU#24, IMU#16,IMU#17 & IMU#8 and calibrated compensated magnetometer data of the IMU#8.

Appendix 2

Sample code (Matlab) for Stepwise Dead Reckoning

%Visit the support page from www.inertialelements.com and check the "Application Note" under MIMU22BL Documents under "Resources" TAB for details about the command.

```
% Reset stepwise deadreckoning INS
fwrite(com, [52 0 52],'uint8');
fread(com,4,'uint8')

package_number_old = nan;
whileabort_flag==0
if(com.BytesAvailable>=64)
header = fread(com,2,'uint8'); % Header + number + Payload size (-) 4, (+)2
payload = fread(com,14,'float');
step_counter = fread(com,1,'uint16'); % Step counter
fread(com,1,'uint16'); % Checksum
fprintf(file,'%i %i %12.9f %12.9f %12.9f %12.9f %12.9f %12.9f %12.9f %12.9f %12.9f %12.9f
%12.9f %12.9f %12.9f\n',step_counter,payload);% for USB
dx = payload(1:4);
dP = Pvec2Pmat(payload(5:14));
pdef=find(eig(dP)<=0);
ifisempty(pdef)
chol(dP,'lower');
end
        [x_swP_sw] = stepwise_dr_tu(x_sw,P_sw,dx,dP);
figure(1)
plot([x_sw(1) x_sw_old(1)],[x_sw(2) x_sw_old(2)],'b');
axis equal;
drawnow;
x_sw_old = x_sw;
P_sw_old = P_sw;
```

```
end
end

% Stop output
fwrite(com, [50 0 50],'uint8'); % Processing off
fwrite(com, [34 0 34],'uint8'); % all output off

---------------------------------------

% stepwise_dr_tu
function [x2_out P2_out] = stepwise_dr_tu(x2_in,P2_in,dx2,dP)

% Input matrix
sin_phi = sin(x2_in(4));
cos_phi = cos(x2_in(4));

dR = [cos_phi -sin_phi 0 0;
sin_phicos_phi 0 0;
    0       0       1 0;
    0       0       0 1];

F = [1 0 0 -sin_phi*dx2(1)-cos_phi*dx2(2);
    0 1 0 cos_phi*dx2(1)-sin_phi*dx2(2);
    0 0 1 0;
    0 0 0 1];

% Update upper layer system
x2_out = x2_in+dR*dx2;

P2_out = F*P2_in*F' + dR*dP*dR';

end
```

Appendix 3

1. **How to ensure integrity of the output data packet by using Checksum**

The last two bytes of the output packets are Checksum, which is 16-bits unsigned integer. MIMU4844/48XC computes checksum by adding rest of the bytes and putting result in the last two bytes of the data packed as unsigned integer. When the data packet is received, Checksum is also extracted from the data packet (except last two bytes), and compared with the last two bytes which are the Checksum computed by the MIMU4844/48XC. The two Checksums must match.

Steps to extract and compute Checksumfrom IMUs data packet are described below

(i) Extracting IMU's Checksum from its output data packet

Below is the method (pseudo code) to obtain last two bytes of the packet and convert to integer:

```
int getChecksum(Byte[] data){
        int size = data.length; // Output data packet size
        int firstByte = data [size-2] & 0xFF; // Saving first byte of checksum as 4-bytes integer
        int secondByte = data [size-1] & 0xFF; // Saving second byte of checksum as 4-bytes integer
        return firstByte*255 + secondByte; // Obtaining 4-bytes integer value of the checksum
}
```

(ii) Computing Checksum from the received data packet

Below is the method (pseudo code) to compute checksum from received data packet:

```
int calculateChecksum(Byte[] data){
    int sum = 0; // 'sum' is 4-bytes integer
    for (int i = 0; i< data.length-2 ; i++) // Add all the bytes of data packet except last two
        sum += data [i] & 0xFF; // Add and accumulate
    return sum%65536; // This is the final checksum
}
```

2. **Format of Command Acknowledgement packet (ACK)**

MIMU4844/48XC transmits 4 bytes Acknowledgement (ACK) packet in response to any command from the application platform. First byte of the ACK packet is always 0xA0, second byte is the first byte of the command (also known as command header) and last two bytes are the Checksum.

For example, following is the 4-bytes ACK packet in response to processing off command [50 0 50].

"0xA0 0x50 0x00 0xF0"

0xA0 = Acknowledgement state (Denotes ACK packet)
0x50 = command header (First byte of command packet)
0x00 0xF0 = Checksum

It is important to note that format of ACK in response to data packet is different than that of in response to command. Sizes of ACK (data) and ACK (command) packets are 5 bytes and 4 bytes respectively.

3. **Format of the Output Header**

The 4 bytes output header contains [0xAA pl1 pl2 pl3]. The significance of each byte is described below:

- 0xAA- Packet Header: This marks the beginning of a new data packet.
- pl1 and pl2 are 2 bytes packet number in 2 byte unsigned integer format. If the packet number exceeds 65535 (i.e overflow occurs) then it will show the packet number modulus 65536 (packet number %65536).

- pl3 is the payload size in single byte unsigned integer format. . If the payload size exceeds 255 (i.e overflow occurs) then it will show the payload size modulus 256 (payload size %256).

4. Format of acknowledge packet in response to data packet (PKG ACK)

PKG ACK is different from ACK. This PKG ACK is send by the application platform to MIMU4844/48XC on successfully receiving a data packet (only if lossless mode is set).

PKG ACK consists of 5 bytes:

i. 1st byte: 01

ii. 2nd byte: P1 (First byte of data packet number of last data packet received)

iii. 3rd byte: P2 (Second byte of data packet number of last data packet received)

iv. 4th byte: Quotient of {(1+P1+P2) div 256}

v. 5th byte: Remainder of {(1+P1+P2) div 256}

Pseudo code for PKG ACK generation:

```
i=0;
for(j=0; j<4; j++)
{
 header[j]=buffer[i++]& 0xFF; // 4-bytes assigned toHEADER; }
 packet_number_1=header[1]; // First byte of data packet number
 packet_number_2=header[2]; // Second byte of data packet number
 ack = createAck(ack,packet_number_1,packet_number_2); // Acknowledgement created
 <code to send acknowledgement> //ACKNOWLEDGEMENT SENT


 // HOW TO CREATE ACKNOWLEDGEMENT
 // ACK consists of 5 bytes
 createAck(byte[] ack, int packet_number_1, int packet_number_2)
 {
        ack[0]=0x01; // 1st byte
        ack[1]= (byte)packet_number_1; // 2nd byte
        ack[2]= (byte)packet_number_2; // 3rd byte
        ack[3]= (byte)((1+packet_number_1+packet_number_2-
(1+packet_number_1+packet_number_2) % 256)/256); // 4th byte – Quotient of {(1+P1+P2) div 256}
        ack[4]= (byte)((1+packet_number_1+packet_number_2) % 256); // 5th byte- Remainder of
{(1+P1+P2) div 256}
        return ack;
 }
```